ICT2104B • OBJECT ORIENTED PROGRAMMING II (JAVA)

MODULE 3: TAKING CONTROL

Sequence · Selection · Iteration · if / switch · while / do-while / for · Input Validation

Duration: 9 Hours | **Builds on:** Module 2 — The Building Blocks
Primary Reference: *Sedgewick & Wayne, Introduction to Programming in Java: An Interdisciplinary Approach, 2nd Ed. (§1.3)*

Learning Outcomes

By the end of this module, you should be able to:

- Differentiate between sequence, selection, and iteration as the three fundamental control-flow structures.
- Use if, if...else, and switch statements to make decisions in a program.
- Use for, while, and do...while loops to repeat actions.
- Select the most appropriate selection or iteration construct for a given programming problem.
- Write simple, robust input-validation routines that reject bad data before it corrupts a program's logic.

3.0 Introduction: From Straight Lines to Real Decisions

Every program you wrote in Module 2 behaved the same way, no matter what: each statement ran exactly once, top to bottom. That works for printing a receipt, but it is not how real software — or real life — actually behaves.

A Mobile Money agent doesn't perform identical steps for every customer — they *decide* what to do based on whether you're sending money, buying airtime, or checking your balance. A SACCO loan officer doesn't stop after checking one applicant — they *repeat* the same eligibility check for every single applicant in the queue. This module gives your programs that same power: the ability to **decide** and to **repeat**.

> 🔑 **KEY POINT**
>
> Programmers call the order in which statements actually execute **control flow**. This module introduces the three control-flow structures every program on Earth is built from: sequence, selection, and iteration.

3.1 Sequence, Selection, and Iteration

Structure	What It Means	Everyday Analogy
Sequence	Statements execute top to bottom, each exactly once.	Following a recipe step by step, in order.
Selection	The program chooses between two or more paths, based on a condition.	“If it's raining, carry an umbrella; otherwise, don't.”
Iteration	A block of statements repeats, either a fixed number of times or until a condition changes.	“Keep frying chapati until the dough runs out.”

Every program you will ever write is built from just these three structures, combined and nested. Here is a compact preview of all three working together — we unpack each one properly in the rest of this module.

Preview.java *java*

```java
int mark = 65;              // SEQUENCE: runs once, in order

if (mark >= 50) {           // SELECTION: choose a path
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}

for (int i = 1; i <= 3; i++) {    // ITERATION: repeat a fixed number
    System.out.println("Reminder " + i);
}
```

Console Output

```
Pass
Reminder 1
Reminder 2
Reminder 3
```

3.2 Making Decisions: if and if...else

The **if** statement lets a program execute a block of code only when a condition — a boolean expression — evaluates to true. Adding an **else** clause gives you a second path to take when the condition is false.

Anatomy of an if...else Statement

Code Fragment	Part	Purpose
if (mark >= 50)	Condition	A boolean expression; the block runs only when this is true.

Code Fragment	Part	Purpose
{ … }	Block (body)	One or more statements to execute for that path.
else { … }	Alternative path	Runs instead, only when the condition is false.

You can chain several conditions together with else if to test more than two possibilities, checked in order from top to bottom until one matches.

Practical Code Example 3.1 — Classifying Your Module 2 Result

Recall the CourseGradeCalculator capstone from Module 2, which computed a total mark of **71.0%**. A grading system still needs to turn that number into a letter grade — exactly the kind of multi-way decision an if…else if chain is built for.

GradeClassifier.java *java*

```java
import java.util.Scanner;

public class GradeClassifier {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your total mark (0-100): ");
        double totalMark = input.nextDouble();

        char grade;

        if (totalMark >= 80) {
            grade = 'A';
        } else if (totalMark >= 70) {
            grade = 'B';
        } else if (totalMark >= 60) {
            grade = 'C';
        } else if (totalMark >= 50) {
            grade = 'D';
        } else {
            grade = 'F';
        }

        System.out.println("Mark: " + totalMark);
        System.out.println("Grade: " + grade);

        input.close();
    }
}
```

Console Output

```
Enter your total mark (0-100): 71.0
Mark: 71.0
Grade: B
```

> ⚠ **COMMON MISTAKE — = versus ==**
>
> A single = is the **assignment** operator; == is the **comparison** operator. Mixing them up inside a condition is one of the most common beginner bugs.
>
> For most types, Java's compiler catches this for you — if (age = 20) fails to compile, because the assignment age = 20 is not a boolean value. But watch out with boolean variables specifically — this one is dangerously legal:

BuggyPaymentCheck.java *java*

```java
boolean isPaid = false;

if (isPaid = true) {        // BUG: assigns true, doesn't compare!
   System.out.println("Fees paid.");
}
// This ALWAYS prints "Fees paid." and silently overwrites isPaid,
// no matter what isPaid held before. Always write isPaid == true,
// or simply: if (isPaid)
```

3.3 Multi-Way Decisions: The switch Statement

When a program must branch among **several discrete, known values of the same variable** — a menu option, a day number, a grade letter — a long if...else if chain works, but a **switch** statement is often clearer.

Practical Code Example 3.2 — A Mobile Money Menu

USSD menus like *165# or *185# are switch statements in disguise: one number typed by the user selects one of several fixed actions.

MobileMoneyMenu.java *java*

```java
import java.util.Scanner;

public class MobileMoneyMenu {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.println("----- MOBILE MONEY MENU -----");
        System.out.println("1. Send Money");
        System.out.println("2. Buy Airtime");
        System.out.println("3. Check Balance");
        System.out.println("4. Pay Bill");
        System.out.print("Choose an option (1-4): ");
        int choice = input.nextInt();

        switch (choice) {
            case 1:
                System.out.println("Enter recipient number and amount...");
                break;
            case 2:
                System.out.println("Enter phone number for airtime...");
                break;
            case 3:
                System.out.println("Your balance is UGX 125,000.");
                break;
            case 4:
                System.out.println("Enter biller code and amount...");
                break;
            default:
                System.out.println("Invalid option. Please try again.");
        }

        input.close();
    }
}
```

Console Output

```
----- MOBILE MONEY MENU -----
1. Send Money
2. Buy Airtime
3. Check Balance
4. Pay Bill
Choose an option (1-4): 3
Your balance is UGX 125,000.
```

> ⚠ **COMMON MISTAKE — the missing break**
>
> Without a `break;` statement, execution **falls through** into the next case instead of exiting the switch. This is almost never what you want:

FallthroughBug.java *java*

```java
int day = 2;
switch (day) {
   case 1: System.out.println("Monday");
   case 2: System.out.println("Tuesday");
   case 3: System.out.println("Wednesday");
}
// Prints BOTH "Tuesday" and "Wednesday" -- once day matches case 2,
// execution falls through every case below it until a break appears.
```

> 💡 **TIP**
>
> Always give every case its own break; unless you deliberately want fallthrough, and always include a default case to catch unexpected values.

3.4 Choosing Between if...else and switch

Use if / if...else when...	Use switch when...
The condition is a range (marks ≥ 50) rather than an exact value.	You're testing exact, known values of one single variable.
The condition involves a boolean expression, e.g. age > 18 && hasID.	You have several equally-likely discrete options (a menu, a day, a grade).
You have only two real outcomes.	You want the clarity of a case-by-case layout instead of a long chain.

3.5 Repeating Actions: Loops

A loop repeats a block of statements, so you don't have to copy-paste the same code over and over. Java gives you three loop constructs, and the difference between them comes down to one question: when is the condition checked, and is the number of repetitions known in advance?

3.5.1 The while Loop

A **while** loop checks its condition **before** every iteration, including the first. If the condition is false to begin with, the body never runs at all — this is called a *pre-test loop*.

Practical Code Example 3.3 — Reaching a Savings Goal

Suppose a student saves toward a goal each month. We don't know in advance how many months that will take — a perfect job for a while loop.

SavingsGoalTracker.java *java*

```java
public class SavingsGoalTracker {
    public static void main(String[] args) {
        double balance = 50000.0;       // UGX, starting savings
        double monthlyDeposit = 75000.0; // UGX, saved each month
        double goal = 500000.0;         // UGX, target amount
        int months = 0;

        while (balance < goal) {
            balance = balance + monthlyDeposit;
            months++;
        }

        System.out.println("Months needed: " + months);
        System.out.println("Final balance: UGX " + balance);
    }
}
```

Console Output

```
Months needed: 6
Final balance: UGX 500000.0
```

3.5.2 The do...while Loop

A **do...while** loop checks its condition **after** each iteration, so the body always runs **at least once** — a *post-test loop*. This is ideal for menus: you must show the menu at least once before there is anything to check.

Practical Code Example 3.4 — A Menu That Keeps Running

Let's wrap Example 3.2's menu in a do...while loop, so it keeps showing itself until the user chooses to exit.

MobileMoneyMenuLoop.java *java*

```java
import java.util.Scanner;

public class MobileMoneyMenuLoop {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int choice;

        do {
            System.out.println();
            System.out.println("----- MOBILE MONEY MENU -----");
            System.out.println("1. Send Money");
            System.out.println("2. Check Balance");
            System.out.println("0. Exit");
            System.out.print("Choose an option: ");
            choice = input.nextInt();

            switch (choice) {
                case 1:
                    System.out.println("Enter recipient and amount...");
                    break;
                case 2:
                    System.out.println("Your balance is UGX 125,000.");
                    break;
                case 0:
                    System.out.println("Thank you for using Mobile Money.");
                    break;
                default:
                    System.out.println("Invalid option, try again.");
            }
        } while (choice != 0);

        input.close();
    }
}
```

Console Output

```
----- MOBILE MONEY MENU -----
1. Send Money
2. Check Balance
0. Exit
Choose an option: 2
Your balance is UGX 125,000.

----- MOBILE MONEY MENU -----
1. Send Money
2. Check Balance
0. Exit
Choose an option: 0
Thank you for using Mobile Money.
```

3.5.3 The for Loop

A **for** loop is the natural choice when you know **exactly how many times** to repeat something. It packs initialization, condition, and update into one tidy line.

Anatomy of a for Statement

Code Fragment	Part	Purpose
int distance = 1	Initialization	Runs exactly once, before the loop starts.
distance <= 10	Boolean condition	Checked before every iteration; loop continues while true.
distance++	Update	Runs after each iteration's body finishes.

Practical Code Example 3.5 — A Boda Fare Table

Recall BASE_FARE and RATE_PER_KM from your Module 2 lab. A for loop is perfect for printing the fare for every distance from 1 to 10 km in one go.

FareTable.java *java*

```java
public class FareTable {
    public static void main(String[] args) {
        final double BASE_FARE = 2000.0;
        final double RATE_PER_KM = 500.0;

        System.out.println("Boda Fare Table");
        System.out.println("---------------");

        for (int distance = 1; distance <= 10; distance++) {
            double fare = BASE_FARE + (RATE_PER_KM * distance);
            System.out.println(distance + " km  ->  UGX " + fare);
        }
    }
}
```

Console Output

```
Boda Fare Table
---------------
1 km  ->  UGX 2500.0
2 km  ->  UGX 3000.0
3 km  ->  UGX 3500.0
4 km  ->  UGX 4000.0
5 km  ->  UGX 4500.0
6 km  ->  UGX 5000.0
7 km  ->  UGX 5500.0
8 km  ->  UGX 6000.0
9 km  ->  UGX 6500.0
10 km  ->  UGX 7000.0
```

3.6 Selecting the Most Appropriate Construct

With five constructs now available — if, if...else, switch, while, do...while, and for — the real skill is choosing the right tool for the shape of your problem. Use this as a quick decision guide.

Selection: if / if...else vs. switch

Situation	Best Construct
Exactly two outcomes (yes/no, pass/fail).	if / if...else
A range of values, or a compound boolean condition.	if...else if
Several discrete, known values of one variable (menu, day, grade).	switch

Iteration: for vs. while vs. do...while

Situation	Best Construct
You know exactly how many times to repeat (10 students, 12 months).	for
The number of repetitions is unknown, and could even be zero.	while
The number of repetitions is unknown, but must run at least once (a menu, “ask again until valid”).	do...while

3.7 Input Validation

Users mistype. They enter letters where numbers are expected, ages of -5, or a Mobile Money PIN that isn't 4 digits. A robust program never trusts input blindly — it **validates** it, and keeps asking until the value is acceptable. This is a direct, practical application of the while loop you just learned.

Practical Code Example 3.6 — Validating an Age

This program combines a while loop, a boolean flag, and Scanner's **hasNextInt()** method, which checks whether the next token *can* be read as an int, *without* consuming it — so a bad entry doesn't crash the program with an error.

AgeValidator.java *java*

```java
import java.util.Scanner;

public class AgeValidator {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int age = 0;
        boolean isValid = false;

        while (!isValid) {
            System.out.print("Enter your age (16-100): ");

            if (input.hasNextInt()) {
                age = input.nextInt();
                if (age >= 16 && age <= 100) {
                    isValid = true;
                } else {
                    System.out.println("Age must be between 16 and 100.");
                }
            } else {
                System.out.println("Please enter whole numbers only.");
                input.next();  // discard the invalid token
            }
        }

        System.out.println("Age accepted: " + age);
        input.close();
    }
}
```

Console Output

```
Enter your age (16-100): abc
Please enter whole numbers only.
Enter your age (16-100): 5
Age must be between 16 and 100.
Enter your age (16-100): 22
Age accepted: 22
```

> ⚠ **COMMON MISTAKE — the loop that never ends**
>
> If you call input.nextInt() on a non-numeric token without first checking hasNextInt(), the program crashes. And if you detect bad input but forget to consume it with input.next(), the **same** bad token is read again next loop — producing an infinite loop that never asks the user anything new.

3.8 Bringing It All Together — Mini Case Study

This final program combines every construct from this module: a do...while menu loop (iteration), an if...else if chain to classify the grade (selection), and the weighted-mark formula from Module 2 (sequence) — all driven by user choices.

Notice we used if...else for the menu here instead of switch — with only two real paths (compute a grade, or exit), Section 3.4's guidance points us toward if...else.

- **ResultProcessor.java** *java*

```
import java.util.Scanner;

public class ResultProcessor {
    public static void main(String[] args) {
        final double PROGRESSIVE_WEIGHT = 0.10;
        final double TEST_WEIGHT        = 0.10;
        final double PRACTICAL_WEIGHT   = 0.20;
        final double EXAM_WEIGHT        = 0.60;

        Scanner input = new Scanner(System.in);
        int choice;

        do {
            System.out.println();
            System.out.println("----- ICT2104B RESULT PROCESSOR -----");
            System.out.println("1. Compute My Grade");
            System.out.println("0. Exit");
            System.out.print("Choose an option: ");
            choice = input.nextInt();

            if (choice == 1) {
                System.out.print("Progressive Assessment mark (/100): ");
                double progressive = input.nextDouble();

                System.out.print("Tests mark (/100): ");
                double tests = input.nextDouble();

                System.out.print("Practical/Presentation mark (/100): ");
                double practical = input.nextDouble();

                System.out.print("Final Exam mark (/100): ");
                double exam = input.nextDouble();

                double totalMark = (progressive * PROGRESSIVE_WEIGHT)
                                 + (tests * TEST_WEIGHT)
                                 + (practical * PRACTICAL_WEIGHT)
                                 + (exam * EXAM_WEIGHT);

                char grade;
                if (totalMark >= 80)      grade = 'A';
                else if (totalMark >= 70) grade = 'B';
                else if (totalMark >= 60) grade = 'C';
                else if (totalMark >= 50) grade = 'D';
                else                      grade = 'F';

                System.out.println("Total: " + totalMark
                                 + "%   Grade: " + grade);
            } else if (choice != 0) {
                System.out.println("Invalid option, try again.");
            }

        } while (choice != 0);

        System.out.println("Goodbye!");
```

```
        input.close();
    }
}
```

Console Output

```
----- ICT2104B RESULT PROCESSOR -----
1. Compute My Grade
0. Exit
Choose an option: 1
Progressive Assessment mark (/100): 80
Tests mark (/100): 70
Practical/Presentation mark (/100): 85
Final Exam mark (/100): 65
Total: 71.0%   Grade: B

----- ICT2104B RESULT PROCESSOR -----
1. Compute My Grade
0. Exit
Choose an option: 0
Goodbye!
```

> 💡 **TRY IT YOURSELF**
>
> This capstone doesn't validate the marks it reads. Lab Task 6 asks you to combine this program with the AgeValidator-style validation pattern from Section 3.7 — that's exactly how real, production-quality input handling is built.

Lab Activity 3 — Taking Control in Practice

Complete the following tasks during your lab session. Save each program as its own .java file using the exact class name given, and be ready to demonstrate your output.

Task 1 — ExamEligibility.java (if…else if)

Write a program that reads a student's attendance percentage (a double) and classifies their exam eligibility using this policy:

- 75% or above: “Eligible for exams.”
- 60% up to (but not including) 75%: “Eligible with a warning letter.”
- Below 60%: “Not eligible — see the Dean of Students.”

Task 2 — Extend the Mobile Money Menu (switch + do…while)

Starting from MobileMoneyMenuLoop.java (Example 3.4), add a new case for “3. Buy Airtime”. The menu must keep reappearing until the user selects 0 to exit.

Task 3 — BodaFareTable.java (for loop)

Reuse the BASE_FARE and RATE_PER_KM constants from your Module 2 lab. Read a maximum distance (an int) from the keyboard, then use a for loop to print the fare for every whole kilometre from 1 up to that maximum, in the same style as Example 3.5.

Task 4 — PinValidator.java (input validation)

Write a program that keeps prompting for a 4-digit Mobile Money PIN until a valid one is entered. Represent the PIN as an int and check that it falls between 1000 and 9999 inclusive, following the validation pattern from AgeValidator.java (Example 3.6), including the hasNextInt() guard.

Task 5 — Debug It

The two snippets below each contain a bug related to this module's concepts. Identify the bug, explain in one sentence why it fails, and rewrite a corrected version.

Snippet A

SnippetA.java *java*

```java
boolean hasPaidFees = false;
if (hasPaidFees = true) {
    System.out.println("Access granted.");
} else {
    System.out.println("Access denied.");
}
```

Snippet B

SnippetB.java *java*

```java
int option = 2;
switch (option) {
    case 1: System.out.println("Deposit");
    case 2: System.out.println("Withdraw");
    case 3: System.out.println("Balance");
}
```

Task 6 — Challenge: Validate the Capstone

Copy ResultProcessor.java (Section 3.8) into a new file, ResultProcessorValidated.java. Wrap each mark-reading prompt in a while loop so that no mark below 0 or above 100 is ever accepted, exactly as AgeValidator.java validates age.

🔑 SUBMISSION CHECKLIST

- ☑ All six .java files compile and run without errors.
- ☑ Every switch case (except intentional fallthrough) ends with break.
- ☑ Every validation loop actually terminates on valid input — test it.
- ☑ Comparisons use ==, never a bare = , inside conditions.

Recap: Key Takeaways

Concept	In One Line
Sequence	Statements run top to bottom, each exactly once — the Module 2 default.
Selection (if / if...else)	Branch between one or two paths based on a boolean condition.
Selection (switch)	Branch among several known values of one variable; never forget break.
Iteration (while)	Pre-test loop: checks first, so it may run zero times.
Iteration (do...while)	Post-test loop: always runs at least once, ideal for menus.
Iteration (for)	Best when the number of repetitions is known in advance.
Choosing a construct	Match the shape of the problem to the construct (Section 3.6).
Input validation	Re-prompt in a loop until a value is both well-formed and in range.

You can now make your programs decide and repeat — the two abilities that turn a straight-line script into real, responsive software. Combined with Module 2's data types and variables, you have everything needed to build genuinely interactive command-line programs.

🔑 LOOKING AHEAD

Module 4 — **Classes and Objects** — introduces true object-oriented programming: what a class and an object actually are, encapsulation, and how to declare objects and call their methods in Java.